

decb(1.0.gamma)

decb(1.0.gamma)

decb MANUAL

SYNOPSIS

decb [options] [filename] [filename] ...

OPTIONS

-c, --capitalize
 Enable capitalization on input
--conditions
 Print GPL β redistribution conditions and exit
--errors-list
 Print a complete errors list and exit
-g, --grace
 Enable grace mode in listing (spacing and indenting)
-h, --help
 Display this information and exit
--insert
 Insert line numbers before each line of file and list
--length
 Print number of characters in the program file
-L [n1[-n2]]
 List file lines from <n1> to <n2>
--list
 List all file lines
-N <name>
 Show in header <name> instead of current file name
-n, --no-header
 Disable printing of header and footer
-R [n[,f,k]]
 Resequence and list file lines (f becomes n, step k)
--resequence
 Resequence and list file lines (default values for f,n,k)
--reverse
 Reverse listing order for option -l
-S <num>
 Run program from line number <num>
--save
 Set save mode for options -r and -i (enable overwriting file)
-T <num>
 Set <num> as the tab stop for list indenting, in grace mode
-v, --version
 Display version and exit
--warranty
 Print warranty conditions from GPL β and exit

A [filename] may have any name or extension, and may be located anywhere in the system tree, provided the whole path is specified; remember that whenever a file is called within a program (with CHAIN or FILES, for instance), the called file must be stored in the same directory, or it must be called along with the whole path. If more than one filename is specified, only the first will be executed; the others will be ignored.

[options] may be input as interspersed single- and double-dash unities; single-dash options may of course be grouped, provided the ones requiring an argument are left as last. Any or no space may be put between any option and its argument, or between arguments.

If you find some option weird, remember this is a program conceived to be run as a BASIC language server to vibes (the MONITOR and the BASIC interface emulator, not yet ready for public).

And... this is not a BASIC primer.

DESCRIPTION

This is decb, a program for interpreting and executing DEC-20 BASIC programs written according to version 17 (mainly 1975-76 version 17F, a direct evolution, with minor issues, of the original 1969 Version V of the mythical Dartmouth BASIC), which ran on DEC machines in the early Seventies over the MONITOR shell; decb is a shortcut for 'DEC BASIC'. References to documentation are in [2], [3] and [4].

The decb BASIC language is a very compliant version of the DEC-20 one; all the restrictions applied "there", are generally applied "here"; all features are inserted, and a BASIC program yields, apart from some minor issues, just the same results and output lines of the original DEC-20.

decb owes all of his BASIC engine to vibes, a program I wrote years ago which emulated the MONITOR, the BASIC interface and a primitive version of the DEC-10 BASIC; vibes, in turn, owes many things to dib, my former emulator for 1966 Dartmouth BASIC Version III [1], from which I grabbed the parser, the basic language statements running engine, the MAT statements and the error detection system; on top of these I promoted the BASIC interpreter to level V. This list won't be complete if I wouldn't quote here Mr. Spinellis, whose 'dds basic' interpreter served as a first engine for my dib. Thanks, Diomidis.

decb is a UNIX program, it's completely free and is distributed according to the GNU Public License version 3. Donations are welcome through Paypal to tbin at libero dot it.

INSTALLING decb

Warning: decb can be installed on UNIX systems only (any); on old Macs (9.x and lesser) and on Windows it isn't even guaranteed to compile; there are many differences between UNIX and not-UNIX systems for it to happen, but if you can, you're welcome.

Installing decb is very easy; access a terminal (real or emulated) and unpack the tgz package into a directory into your path; decb will unpack into a directory with the name containing version info, like decb.1.0.beta, or something like that. Enter into this directory (let's call it decb/package/). Now, type 'make'; if you have gcc 3.x/4.x installed, it should compile without problems; if you don't have the gcc, you probably have to correct the sources where your compiler points to, but in general it's C for plain gcc.

After compilation, install the executable. If you want to install for every user, turn to root or sudoer, and type 'make install' (root) or 'sudo make install' (sudoer); that's all. If you want to install into your path only, change the destination lines in the makefile and type 'make install' (no need to become root).

After installation, just to be sure decb is up and available, type decb -h or decb -v, to see the help or the version banners. If you see them, you're ready to go. Start recollecting your BASIC skills!

OPTIONS

Options are self explanatory, but I guess some words won't hurt. Apart from legal and informative options (which need no comment), here's a survey about the other available options:

-c, --capitalize (capitalize on input)

Usually, input by keyboard is not changed by decb: if you type a string, for instance, with lower and upper letters, you can do it, and it will follow its path, when processed or printed, conserving intact the original aspect.

If you want, instead, be 'conservative', emulating the behavior of the Early Seventies machines (which dealt with capital letters only), you can enable option -c/--capitalize, which will return all typed text

capitalized. This option is available as a PRAGMA option.

--errors-list (print a list of the system error messages)

This option is useful only if you want to know the error codes associated with the error messages, or to see the list of all the error messages present in the system. It can be typed also as --errors and as --errorslist. A complete list is anyway into this document (see ahead).

-g, --grace: see option -l

--insert (insert line numbers)

Occasionally, you may need to convert a program written for another BASIC, a program, say, with no line numbers, and only labels for GOTOs and GOSUBs. If it's a long program, you'd probably spend much of your time in inserting line numbers one by one. This option does this for you, with lines starting from 10 and with step 10, breaking also multiple instructions on the same line, if separated by colon (:) or backslash (\) and updating inner references (GOTOs, GOSUBs, etc.); options -t, -g/--grace, --reverse and -s/--save have effect on this option. It can be typed also as --insert-line-numbers.

Beware: this algorithm cannot work in case of an IF..THEN followed by an instruction and not by a line reference: this must be done manually; look at the following example:

```
IF A=0 THEN PRINT "WOW!" : B=0 : C=24 : PRINT "OK!"
```

is converted by option --insert into:

```
10 IF A=0 THEN PRINT "WOW!"
20 B=0
30 C=24
40 PRINT "OK!"
50 END                                ' END ADDED IF MISSING
```

After this operation, line 10 should be manually changed into:

```
10 IF A<>0 THEN 50                    ' INVERT CONDITION TEST
```

and a new line should be added manually, containing the statement after THEN, like this:

```
15 PRINT "WOW!"
```

After all these corrections, run the -R/--resequence option, and your file is ready and converted.

Finally, if your program bears instructions like GOTO I*20, put your mind at rest: this cannot be converted, because decb (and the DEC-10 BASIC) cannot resolve formulas as number references, because the variables are instantiated at runtime, and are not known by option --insert.

--length

This command emulates the LENGTH command, returning the length of characters of the program passed to it, before exiting. The length is reported as multiple of 5 units, just as the original LENGTH command, but the results may vary between the two, due to the different used algorithms. It can be also typed as --len.

-L [n1[-n2]], --list

This is a very nice feature, trying to emulate the LIST command. If enabled, it will cause decb to list the program, rather than executing it, but it will print the lines as in memory. So, no special markers or the like will appear, but all regular comments and even the void lines. The n1 and n2 range delimiters may be omitted, and thus the whole program will be listed (as in the case of --list); if you use the sole n1, only that line will be shown; if you use the figure n1-, the listing will proceed from n1 included to the end; if you use the figure -n2, the listing will proceed from start to n2, included; if you use the figure n1-n2, the listing will proceed from n1 included to n2 included. If you prefer a reverse (even partial) listing, as for the LISTREVERSE

and the LISTNHREVERSE commands, use option --reverse. If you want the file to be saved onto the same file, use option --save. (cautions! This will overwrite the previous version!)

The default listing shows the text just as it appears in the source. If you want more appeal, you can activate the -g/--grace option, which will activate three main features: 1) indenting, getting rid of the previous one, 2) spacing, separating statements from their arguments, and 3) featuring the line numbering just as it appears in the MONITOR of the DEC-10, in five digits followed by a tab.

The indenting is three characters long by default. You can control this width by means of option -T <num>, which will set the tab to <num>, in the range 0-16; other values will be ignored and a warning will be shown.

-N <name>

Normally, in the heading of the program, along with date and hour, the name of the program is reported (without extension); if you want to read another name or to control this feature, you can use option -N followed by a name; if this name contains spaces or other special characters, enclose the name into quotes.

-n, --no-header

This option tends to emulate the RUNNH command, disabling the printing of the header and the footer, making the output tighter. It can be typed also as --no-heading and --no-footer.

--no-reset

This option controls the behavior of the memory treatment in CHAIN calls; the DEC-20 reset the memory between a CHAIN call, so that variables instantiated in the first program were not available in the second; some compilers, though, didn't erase memory, so that variables could be shared between multiple programs run via the CHAIN statement. This option enables this sharing. This option is available as a PRAGMA option.

-R [n[,f,k]]

This option tends to emulate the RESEQUENCE command, causing the listing to be renumbered from line <f>, which will receive new number <n>, and the following lines with step <k>. The inner references of GOTO, THEN, GOSUB, USING and the like will be updated too.

You can omit all the arguments, and in this case, the default values for <f>, <n>, <k> will be used (see --resequence). If you want to specify <n> only, you can omit <f> and <k>; If you want to omit <f> or <k>, you should maintain the commas. The default value for <f> is always the first existing available line number, and the default value for <k> is 10.

--resequence is the double-dash version of -R, with default values for any of the three parameters (setting <n> equal to <f>). It can be typed also as --reseq, --renumber and --renum.

The output of the new resequenced listing will not overwrite the existing one, but it's simply printed on the terminal you use. If you want to make this change permanent, you can use option --save, which will cause the listing to overwrite the previous one. Use this last option with care. Finally, options --reverse, -T and -g/--grace have effect on resequencing, and can be used as well.

--reverse: see option -l

-S <num>

This option tends to simulate the RUN command, which accepted <num> as argument, specifying the starting line from which begin the program execution.

--save: see option -l

-T <num>: see option -l

The language understood by decb is thoroughly described in [2] and in [4]. Statements may be written in lower or upper or mixed letters, decb is case insensitive.

Math and strings operators

+	- unary operator for positive numbers (optional)
+	- addition and string concatenation
-	- unary operator for negative numbers
-	- subtraction
*	- multiplication
/	- division
^	- power (classic form)
**	- power (alternating form)

Relational operators

=	- equal
<>	- not equal
<=	- lesser than or equal
<	- lesser than
>=	- greater than or equal
>	- greater than

Statements

:	- formatter image for USING
CHAIN	- transfer control to another BASIC file
CHANGE	- turn a string to its ASCII values and reverse
DATA	- values/strings separated by commas for READ
DEF FN	- define a function, on one or multiple lines
DIMENSION/DIM	- array dimensioning (one or two dimensions)
END	- terminate program (required)
FILE#	- assign sequential files channels
FILE:	- assign random access files channels
FILES	- assign channels files before program execution
FOR..TO..STEP	- loop (also FOR..TO..BY)
GOSUB	- go to a subroutine; return after RETURN
GOTO/GO TO	- go to a specified address
IF END#	- test for end of sequential files
IF END:	- test for end of random access files
IF..THEN/GOTO	- if condition is true, go to a specified address
INPUT	- input values/strings from keyboard
INPUT#	- input values/strings from sequential files
INPUT:	- input values/strings from random access files
LET	- assignment for values/strings (optional)
MARGIN	- set output margin to channels files
MAT + - *	- summing, subtracting, multiplying matrices
MAT CON	- set a matrix to all ones
MAT IDN	- set a square matrix to identity matrix
MAT INPUT	- input matrix values from keyboard
MAT INV	- invert a matrix
MAT PRINT	- print a matrix on screen
MAT READ	- input matrix values from DATA statements
MAT TRN	- transpose a matrix
MAT ZER	- set a matrix to all zeroes
NEXT I,J,K..	- FOR loop terminator (handles multiple variables)
ON..THEN/GOTO	- indexed jump
PAGE/NOPAGE	- set/unset output pages size
PRINT	- print on screen
PRINT#	- print onto sequential files
PRINT:	- print onto random files
QUOTE/NOQUOTE	- set/unset quotes mode
RANDOMIZE	- calculate a new seed for random numbers
READ	- input values/strings from DATA statements
READ#	- input values/strings from sequential files
READ:	- input values/strings from random access files
REM/REMARK/'	- comment until end of line (' also inline)
RESTORE * \$	- reset pointer of DATA numbers/strings
RESTORE#	- set sequential files to beginning for READ

RESTORE:	- set random access files to first item
RETURN	- return from a subroutine invoked by GOSUB
SCRATCH#	- erase a sequential file and set for write
SCRATCH:	- erase a random access file
SET:	- set record pointer for random access files
STOP	- anticipated termination of a program
USING	- introduce a reference for a string formatter
WRITE#	- write onto sequential files
WRITE:	- write onto random access files

Functions

<PA>	- a PRINT specifier to clear screen or page
ABS	- return absolute value
ASC	- return ASCII code of argument
ATN	- return arc tangent (return radians)
CHR\$	- return char corresponding to ASCII code argument
CLOG/LOG10	- return common logarithm (base 10)
COS	- return cosine (argument in radians)
COT	- return cotangent (argument in radians)
DET	- yield determinant after an inversion
EXP	- return exponential (base e)
INSTR	- return the substring of a given string
INT	- return the integer part of a number (floor)
LEFT\$	- return left substring of a given string
LEN	- return string length
LOC	- return number of current record in file
LOF	- return number of last record in file
LOG/LN/LOGE	- return logarithm (base e)
MID\$	- return a substring
NUM	- yield number of values input by MAT INPUT
RIGHT\$	- return a substring from the right
RND	- return random number
SGN	- return sign of argument
SIN	- return sine (argument in radians)
SPACE\$	- return a string of spaces
SQR/SQRT	- return square root
STR\$	- return a string from a numeric value
TAN	- tangent (argument in radians)
TIM	- return number of seconds since program start
VAL	- return a value from a string

Special decb markers

#	- ignore current line if it begins with it
&	- stop loading a program if current line begins with it until next line also beginning with it
@	- stop loading a program if current line begins with it
REM PRAGMA	- the PRAGMA statement

"IGNORE LINE" - If the character '#' is found as the first character of a line, the line will not be loaded and run at all, and any information there contained won't be part of the BASIC execution. In practice, it is a special marker for 'Ignore Current Line'. This lets you store specific comments to the source, or build BASIC programs that can act as executables, if you have the first line run like this: #!\path\to\decb \$1.

"SUSPEND PARSING" - If the character '&' is found as the first character of a line, the file loading is suspended, and following lines are read and discarded until another line beginning with '&' is found (and discarded too), restarting file loading with next line. In practice, it is a special marker for "Suspend Parsing Program". This lets you store any free form text within the source file itself, acting as temporary documentation of the source, or isolate parts of code not to be executed. Useful for comments at the beginning of the source.

"STOP PARSING" - If the character '@' is found as the first character of a line, the file loading will stop, and any information thereafter

contained won't be part of the BASIC execution. In practice, it is a special marker for 'Stop Parsing Program'. This lets you store any free form text in the source file itself, acting as documentation of the source, or you can store the output of the program execution without preventing its re-execution.

"SOURCE BLANKS" - A line that begins with any trailing space after the line number, is stored but not executed, just as in the original DEC-10; options --list will retain such lines, making the listing more attractive.

"PRAGMA" - Let's now discuss the PRAGMA statement; it can be enabled after a REM or REMARK statement, like this:

```
10 REM PRAGMA <args>
```

where <args> is a list of characters, each connected with a special function to be enabled or disabled on runtime. The PRAGMA statement must be used only once and preferably (but not obligatorily) as the very first line. This statement is a REM statement, and thus is treated as a comment by any other BASIC compiler or interpreter; its peculiar usage is to distribute source files (.BAS programs) that can be executed by decb without the need for the user to enable this or that option in order to fulfill the program need (e.g. suppose the original listing expects only upper case letters, you should run the -c option from the command line, but this is not known by the user the first time he sees the file, and even if he peeps at the source he may not realize if such option is to be used; the PRAGMA statement fulfills this need automatically and it makes the program independent of the command line option).

Each argument has to be followed by the letter E for 'enable' or D for 'disable'.

Available PRAGMA arguments are:

```
C    capitalization on input
R    reset memory on CHAIN
```

Examples are (blanks are ignored):

```
10 REM PRAGMA CE RD
10 REMARK PRAGMA C D R E
```

Errors list

Here's the list of codes and error messages hardwired into decb. A number of them are not implemented yet (due to current program architecture), some (surprisingly not listed in the LBMAA-A-D) were grabbed by the DEC at the Living Computer Museum, some were added to support the added features or the specific decb architecture.

In the following, the %s figure means that some words (strings) are substituted during the message printing at runtime, the %d figure means that a number is substituted.

```
# 4 ? MEMORY ERROR (YOU MAY NOT OVERWRITE LINES OR CHANGE THEIR ORDER)
# 5 ? MEMORY ERROR (LINE NUMBERS MAY NOT EXCEED 99999)
# 10 ? FILE %s NOT FOUND
# 13 ? MISSING LINE NUMBER FOLLOWING LINE %d
# 22 ? BAD DATA INTO LINE %d
# 23 ? DATA IS NOT IN CORRECT FORM
# 24 ? END IS NOT LAST
# 25 ? EOF
# 26 ? FAILURE ON ENTRY
# 27 ? FNEND BEFORE DEF
# 28 ? FNEND BEFORE NEXT
# 29 ? FOR WITHOUT NEXT
# 30 ? GOSUB WITHIN DEF
```

31 ? FUNCTION DEFINED TWICE
 # 32 ? ILLEGAL ARGUMENT FOR ASC FUNCTION
 # 33 ? ILLEGAL CHARACTER
 # 34 ? ILLEGAL CONSTANT
 # 35 ? ILLEGAL FORMAT
 # 36 ? ILLEGAL FORMAT WHERE THE WORDS THEN OR GO TO WERE EXPECTED
 # 37 ? ILLEGAL FORMULA
 # 38 ? ILLEGAL INSTRUCTION
 # 39 ? ILLEGAL LINE REFERENCE
 # 40 ? ILLEGAL LINE REFERENCE %d
 # 41 ? ILLEGAL RELATION
 # 42 ? ILLEGAL VARIABLE
 # 43 ? INCORRECT NUMBER OF ARGUMENTS
 # 44 ? INITIAL PART OF STATEMENT NEITHER MATCHES A STATEMENT KEYWORD
 NOR HAS A FORM LEGAL FOR AN IMPLIED LET--CHECK FOR MISSPELLING
 # 45 ? MIXED STRINGS AND NUMBERS
 # 46 ? NESTED DEF
 # 47 ? NEXT WITHOUT FOR
 # 48 ? %d IS NOT AN IMAGE
 # 49 ? NO CHARACTERS IN IMAGE
 # 50 ? NO DATA
 # 51 ? NO END INSTRUCTION
 # 52 ? NO FNEND FOR DEF FN%c
 # 53 ? OUT OF ROOM
 # 54 ? RETURN WITHIN DEF
 # 55 ? SPECIFIED LINE IS NOT AN IMAGE
 # 56 ? STRING RECORD LENGTH > 132 OR < 1
 # 57 ? STRING VECTOR IS 2-DIM ARRAY
 # 58 ? SYSTEM ERROR
 # 59 ? TOO MANY FILES
 # 60 ? UNDEFINED FUNCTION -- FN%c
 # 61 ? UNDEFINED LINE NUMBER %d
 # 62 ? USE VECTOR, NOT ARRAY
 # 63 ? VARIABLE DIMENSIONED TWICE
 # 64 ? VECTOR CANNOT BE ARRAY
 # 65 ? %s WAS SEEN WHERE %s WAS EXPECTED
 # 66 % ABSOLUTE VALUE RAISED TO POWER
 # 67 ? ATTEMPT TO OUTPUT A NEGATIVE NUMBER TO A * OR \$ FIELD
 # 68 ? ATTEMPT TO OUTPUT A NUMBER TO A STRING FIELD OR A STRING TO A
 NUMERIC FIELD
 # 69 ? ATTEMPT TO READ# OR INPUT# FROM A FILE WHICH DOES NOT EXIST
 # 70 ? ATTEMPT TO READ# OR INPUT# FROM A FILE WHICH IS IN WRITE# OR
 PRINT# MODE
 # 71 ? ATTEMPT TO WRITE A LINE NUMBER > 99,999
 # 72 ? ATTEMPT OT WRITE# OR PRINT# TO A FILE WHICH HAS NOT BEEN
 SCRATCH#ED
 # 73 ? ATTEMPT TO WRITE# OR PRINT# FROM A FILE WHICH IS IN READ# OR
 INPUT# MODE
 # 74 ? CHR\$ ARGUMENT OUT OF BOUNDS
 # 75 ? DATA FILE LINE TOO LONG
 # 76 ? DIMENSION ERROR
 # 77 % DIVISION BY ZERO
 # 78 ? EXPONENT REQUESTED FOR * OR \$ FIELD
 # 79 ? FILE IS NOT RANDOM ACCESS
 # 80 ? FILE NEVER ESTABLISHED -- REFERENCED
 # 81 ? FILE NOT FOUND BY RESTORE COMMAND
 # 82 ? FILE %s ON MORE THAN ONE CHANNEL
 # 83 ? FILE NOT IN CORRECT FORM
 # 84 ? FILE RECORD LENGTH OR TYPE DOES NOT MATCH
 # 85 ? IF END ASKED FOR UNREADABLE FILE
 # 86 ? ILLEGAL CHARACTER IN STRING
 # 87 ? ILLEGAL CHARACTER SEEN
 # 88 ? ILLEGAL FILENAME
 # 89 ? ILLEGAL LINE REFERENCE IN CHAIN
 # 90 ? IMPOSSIBLE VECTOR LENGTH
 # 91 ? INPUT DATA NOT IN CORRECT FORM--PLEASE RETYPE
 # 92 ? INSTR ARGUMENT OUT OF BOUNDS
 # 93 ? LEFT\$ ARGUMENT OUT OF BOUNDS
 # 94 ? LINE NUMBER OUT OF BOUNDS
 # 95 % LOG OF NEGATIVE NUMBER


```

# 96 % LOG OF ZERO
# 97 % MAGNITUDE OF SIN OR COS TOO LARGE TO BE SIGNIFICANT
# 98 ? MARGIN OUT OF BOUNDS
# 99 ? MARGIN TOO SMALL
#100 ? MID$ ARGUMENT OUT OF BOUNDS
#101 ? MIXED RANDOM & SEQUENTIAL ACCESS
#102 ? MIXED READ#/INPUT#
#103 ? MIXED WRITE#/PRINT#
#104 ? NEGATIVE STRING LENGTH
#105 ? NO FIELDS IN IMAGE
#106 ? NO ROOM FOR STRING
#107 ? NO SUCH LINE IN RUN (NH) OR CHAIN
#108 ? NOT ENOUGH -- ADD MORE
#109 ? ON EVALUATED OUT OF RANGE
#110 ? OUT OF DATA
#111 ? OUTPUT ITEM TOO LONG FOR LINE
#112 ? OUTPUT LINE > 132 CHARACTERS
#113 ? OUTPUT STRING LENGTH > RECORD LENGTH
#114 % OVERFLOW
#115 % OVERFLOW IN EXP
#116 ? PAGE LENGTH OUT OF BOUNDS
#117 ? QUOTA EXCEEDED OR BLOCK NO. TOO LARGE ON OUTPUT DEVICE
#118 ? RETURN BEFORE GOSUB
#119 ? RIGHT$ ARGUMENT OUT OF BOUNDS
#120 ? SET ARGUMENT OUT OF BOUNDS
#121 % SINGULAR MATRIX INVERTED
#122 ? SPACE$ ARGUMENT OUT OF BOUNDS
#123 % SQRT OF NEGATIVE NUMBER
#124 ? STRING FORMULA > 132 CHARACTERS
#125 ? STRING RECORD LENGTH > 132 OR < 1
#126 ? SUBROUTINE OR FUNCTION CALLS ITSELF
#127 % TAN OF PI/2 OR COTAN OF ZERO
#128 ? TOO MANY ELEMENTS -- RETYPE LINE
#129 % UNDERFLOW IN EXP
#130 % UNDERFLOW
#131 ? VAL ARGUMENT NOT IN CORRECT FORM
#132 % ZERO TO A NEGATIVE POWER
#176 ? GOSUB NESTED TOO DEEPLY
#179 ? CHANNEL NUMBER IS <1 OR >9
#180 ? LINE TOO LONG
#190 ? OUT OF MEMORY WHILE DIMENSIONING AN ARRAY
#191 ? OUT OF MEMORY WHILE DEALING WITH VARIABLES
#192 ? OUT OF MEMORY WHILE DEFINING DEF FN%c FUNCTION
#193 ? OUT OF MEMORY WHILE STORING DATA STRINGS
#194 ? OUT OF MEMORY WHILE STORING LINE %d
#195 ? %s IS A DIRECTORY -- CANNOT EXECUTE
#196 ? MISSING FILE NAME
#197 ? CANNOT ACCESS FILE %s
#198 ? OUT OF MEMORY FOR NESTED GOSUB
#199 ? PRAGMA STATEMENT MUST BE USED ONCE ONLY
#200 ? MISSING NAME FOR OPTION -N
#201 % VALUE OUT OF BOUNDS FOR OPTION -t: SETTING DEFAULT
#202 ? UNRECOGNIZED OPTION -%c
#203 ? MISSING %s ARGUMENT FOR OPTION -%c

```

Error messages have been accurately transcribed from the LBMAA-A-D errors list, and where you may find differences is where I changed the text to reflect decb specific behaviors or to match the exact text reported by the DEC-10 at the Living Computer Museum. See the file errors.h for some notes about errors.

Holes in enumeration of error messages reflect either error messages of the MONITOR (the interface of the OS in DEC-10/20), which were part of the original version of vibes (my MONITOR emulator, including a primitive version of BASIC), and which were deleted in decb because useless, or holes I reserved for future enhancements.

Not all the error messages have been used. For a very small number of them, I wasn't able to detect their meaning or to conceive a way to insert them, yet. Oh my!.

A BIT OF LOGIC IN BASIC

Some DEC-20 BASIC coeval brothers had some operators dealing with logic (for instance the OSU Basic). This turned to be a standard after the Microsoft QuickBasic appeared on the market, but the DEC-20 BASIC couldn't make use of them, anyway, because they were not implemented in it.

Here's the scope of this chapter: making full use of the math abilities of the DEC-20 BASIC implemented into decb, in the logic area, for the sake of logic itself. Of course, all the following considerations must not be considered in the bitwise area.

Remember that probably the proposed solutions won't be applicable in the DEC-20 BASIC, because the relational operators returns 'flags' that are not considered 'numbers', and thus cannot be used into formulas. But decb can, and so here are some advices about its usage.

Mybe you wonder why such a chapter has been put here: logic is my favorite math discipline, no surprise then.

The logic operators

The logic infix operators are commonly AND, OR, XOR, EQV, IMP, NOR, NAND; another important logic prefix operator is NOT.

Here are the logic truth tables of the prefix operators. A and B are the results of comparison tests (usually returning -1 in case of true, and 0 - zero - in case of false); for instance, for A we can use $X > 0$, and for B we can use $Y < 24$

A	B	AND	OR	XOR	EQV	IMP	NOR	NAND
-1	-1	-1	-1	0	-1	-1	0	0
-1	0	0	-1	-1	0	0	0	-1
0	-1	0	-1	-1	0	-1	0	-1
0	0	0	0	0	-1	-1	-1	-1

The logic math

Now, let's consider the math operators + (addition), * (multiplication) and - (subtraction). Division is not in the group, because of its weird tendency to fail if you try to divide by 0.

A	B	+	*	-
-1	-1	-2	1	0
-1	0	-1	0	-1
0	-1	-1	0	1
0	0	0	0	0

It's easy to see that the following conditions hold:

Logic operators		Logic math	Example
A AND B	==>	$A * B > 0$	$(X > 0) * (Y < 24) > 0$
A OR B	==>	$A + B < 0$	$(X > 0) + (Y < 24) < 0$
A XOR B	==>	$A - B = 0$	$(X > 0) - (Y < 24) = 0$
A EQV B	==>	$A - B < > 0$	$(X > 0) - (Y < 24) < > 0$
A IMP B	==>	$A - B = 0$	$(X > 0) - (Y < 24) = 0$
A NOR B	==>	$A + B = 0$	$(X > 0) + (Y < 24) = 0$
A NAND B	==>	$A * B = 0$	$(X > 0) * (Y < 24) = 0$

These operators cannot be also used with plain values. Look at this:

-1 AND 0	==>	$-1 * 0 > 0$	returns 0, ok
-1 AND -1	==>	$-1 * -1 > 0$	returns -1, ok
24 AND 0	==>	$24 * 0 > 0$	returns 0, ok as well, but

14 AND -7 ==> 14*-7>0 returns 0

The last result is to be interpreted now bitwise, but logically: both 14 and -7 are true values, because they are not null. So I expect that 'true AND true = true', but it's not.

The same may be said for operators involved with sum and subtraction.

The conclusion is: math logic is strictly bound to logical truth values.

The NOT prefix operator

While the infix operators present an easy math, the NOT operator is rather a different matter, since BASIC has no operator to turn a truth value to its opposite.

Let's begin with the truth table:

A	NOT
-1	0
0	-1

Now let's consider math operations; which? Addition and subtraction, since multiplication and division, at most, amplify or reduce an effect. The operation will be sum and subtraction with -1 before and after.

A	-1	+1	1-	1+
-1	-2	0	2	0
0	-1	1	1	1

Addition gives the correct result in both cases, but only ****if**** A is a truth value, that is if it's -1 or 0. This method **DOESN'T** work with any positive true value.

Nonetheless it can be used with comparison operators:

NOT A ==> A+1 or 1+A (X>0)+1 or 1+(X>0)

The same conclusion here: math logic is strictly bound to logical truth values.

TRIVIA

decb.vim

In the package there's a file called decb.vim, which may help vim/gvim users to write decb BASIC programs with a colored-syntax sized for PDP-10 BASIC source files. Follow these instructions in order to install it:

1. Look for the file filetype.vim in the user vim directory (Programs/vimfiles/ for Windows and \$HOME/.vim/ for UNIX), open it with a text editor and add the following lines to it (create this file if it doesn't exist):

```
" decb (DEC-SYSTEM BASIC)
au BufNewFile,BufRead *.BAS setf decb
```

2. copy decb.vim into syntax/; you may create this directory if it does not exist (Programs/vimfiles/syntax for Windows and \$HOME/.vim/syntax for UNIX).

3. You're done. Now, when you open a .BAS file with vim you will see the proper statements colored by function. Happy programming.

diff.txt

In the package there's a file called diff.txt, which may be consulted for all the differences in the behavior of the DEC-10 in confront with decb. Mainly, they deal with different capabilities of the machines, but sometimes they reflect my poorness in programming. Forgive me.

SEE ALSO

Some documents and references played an important role during the design and coding of decb, and many of them were consulted even earlier, during the making of dib [1]; in the following, some http links may be split to make them fill up the 80-columns space, so recompose them before using them into a browser.

- [1] dib (dib is BASIC) , the emulator of version III (1966) of the language (another program of mine belonging to the deer project) is available here: <http://digilander.it/tonibin/deer/dib>

- [2] BASIC User's Guide" [LBMAA-A-D], a printed book (in pdf form) for the DEC-20, version 17F (with minor differences from previous, with updated examples and some mistake too!) is freely available at <http://bitsavers.informatik.uni-stuttgart.de/www.computer.museum.uq.edu.au/pdf/DEC-20-LBMAA-A-D%20BASIC%20User%27s%20Guide.pdf>

The "BASIC User's Guide" is a reprint of the book "BASIC Conversational Language Manual" [LBLMA-A-D], the older DEC-10 BASIC reference for the BASIC language and interface, also freely available as pdf at http://www.bitsavers.org/pdf/dec/pdp10/TOPS10_softwareNotes/books/vol06/DEC-10-LBLMA-A-D_BASIC.pdf

A Reference card in pdf for version 17B (a rapid consultative document) can be freely found at [http://bitsavers.informatik.uni-stuttgart.de/www.computer.museum.uq.edu.au/pdf/DEC-10-XBRCA-A-D%20DEC-system10%20Basic%20Language%20Reference%20Card%20\(Version%2017B\).pdf](http://bitsavers.informatik.uni-stuttgart.de/www.computer.museum.uq.edu.au/pdf/DEC-10-XBRCA-A-D%20DEC-system10%20Basic%20Language%20Reference%20Card%20(Version%2017B).pdf)

These texts were the basis of my study about decb creation.

- [3] Programming with BASIC, by Byron S. Gottfried, edition 1, 1975 (Italian version, as Programmare in BASIC, Schaum 1982)

This book describes a slightly different BASIC version than Dartmouth version V or DEC version 17B/D/F. This was my first book about programming!

- [4] BASIC Sessions, by Thomas E. Kurtz, chapter XI of the book History of programming languages

The article, about the original BASIC environment, 1977 ca, was once freely available, and I downloaded it, both as a typewriter transcript (titled BASIC) and as chapter XI of the issued volume, but now ACM has retired the free availability imposing a fee to download it. Shame on it!

- [5] BASIC Version 2. revision B

The CDC manual for the BASIC that ran on Cyber 70/170 and 6000 Series computers in 1974, freely available as pdf at http://bitsavers.informatik.uni-stuttgart.de/pdf/cdc/cyber/lang/basic/19980300B_BASIC_Language_Version_2_Reference_Nov74.pdf, was also useful for the making of decb.

- [6] BASIC's User Manual

The Oregon State University manual for the BASIC that ran on a Control Data 3300 computer in 1972, freely available as pdf at <http://bitsavers.informatik.uni-stuttgart.de/pdf/oregon->

State/os3/CCM-71-08_BASIC_May72.pdf, was also useful for the making of decb.

MORE...

Dartmouth BASIC II manual

Manual for Dartmouth BASIC version II, October 1964, freely available as pdf at http://www.bitsavers.org/pdf/dartmouth/BASIC_Oct64.pdf

Dartmouth BASIC III manual

Manual for Dartmouth BASIC version III, June 1965 (rev. September 1966, printed by General Electric), freely available as a pdf file at www.bitsavers.org/pdf/ge/MarkI_Timesharing/IPC-202026_BASIC_Sep66.pdf

Dartmouth BASIC IV manual

Manual for Dartmouth BASIC version IV, January 1968, freely available as a pdf file at http://www.bitsavers.org/pdf/dartmouth/BASIC_4th_Edition_Jan68.pdf

John Kemeny

News about the main inventor of BASIC, prof. John Kemeny, may be found at <http://cis-alumni.org/JKemeney.html>. A pdf rendition may be downloaded from <TODO>.

Thomas Kurtz

News about the co-inventor of BASIC and strict Kemeny's friend and colleague, Thomas Kurtz, may be found at <http://cis-alumni.org/TKurtz.html>. A pdf rendition may be downloaded from <TODO>.

Mean Matrices

This article by Valentin Albillo on ill-conditioned matrices appears on Vol. 24 of the HPCC Club Magazine, and was once available as a pdf file at <http://membres.lycos.fr/albillo/calc/pdf/DatafileVA014.pdf> but it's no more, and I cannot find it elsewhere. Sorry.

Zero to the Zero Power

by Su, Francis E., et al., in Mudd Math Fun Facts, <http://www.math.hmc.edu/funfacts/ffiles/10005.3-5.shtml> where you can find an explanation to the fact that zero to the zero power is 1. Check also <http://mathforum.org/dr.math/faq/faq.0.to.0.power.html> where it's stated: "Consensus has recently been built around setting the value of $0^0 = 1$ ".

The Living Computer Museum

offers a free access to the mainframes in their possessions, among whose a DEC-10 system is available. I used it extensively to test some specific behaviors, but beware: it's often off-line, due to maintenance or to bugs.

BUGS

Known bugs are listed below. They are likely to be fixed with next versions.

1.

Address bugs you find to <tb in at libero dot it>; include the Operating System identification with version, the gcc version you used to compile, the program you were running, the extract of the execution (if you can copy from the terminal) in a txt file, and/or a brief description of what you were about to do and what you got instead. English or

Italian, please. A mixture will do, of course! ;-)

HISTORY

decb appeared in 2014 for the first time. Written by Antonio Maschio, engineer in Montebelluna (TV), Italy.

Antonio Maschio

2009-2014

decb(1.0.gamma)